ADVANCE NLP (AML23702)

LAB EXERCISE: 1

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Experiment Title	Information Retrieval-Based Question Answering using TF-IDF and Cosine Similarity

Aim / Objective

Purpose: *States what the experiment sets out to build and why.*

This experiment develops a simple information retrieval-based Question Answering (QA) system in Python. It accepts a question, compares it with a small collection of documents or sentences, and returns the sentence most relevant to the question. TF-IDF converts text into weighted numerical vectors, while cosine similarity measures similarity between the question and each candidate sentence.

Expected Outcomes Addressed

Purpose: *Maps the implementation to the expected practical outcomes.*

- **Outcome 1: Prepare a text corpus** — Create a collection of topic-related sentences.
- **Outcome 2: Preprocess text** — Normalize case and handle punctuation where appropriate.
- **Outcome 3: Represent text numerically** — Use TF-IDF to weight informative terms.
- **Outcome 4: Retrieve the best answer** — Compute cosine similarity and select the highest-scoring sentence.
- **Outcome 5: Evaluate results** — Test questions and discuss limitations.

Tools & Technologies Used

Purpose: *Lists the technical tools used.*

Category	Tool / Library	Version	Purpose
Language / Runtime	Python	3.x	Core implementation
Text processing	scikit-learn	Compatible release	TF-IDF vectorization and cosine similarity
Notebook	Jupyter / Google Colab	—	Run code and display results
Interface	Command-line input	—	Accept a question and show the retrieved answer

Dataset / Corpus Description

Purpose: *Describes the text data used for testing.*

The system uses a small manually prepared corpus about Natural Language Processing and common NLP concepts. It is a demonstration corpus rather than a large benchmark dataset.

ID	Sample sentence
D1	Natural Language Processing enables computers to analyse and understand human language.
D2	Tokenization splits text into smaller units called tokens.
D3	Part-of-speech tagging assigns grammatical labels such as noun, verb, adjective, and adverb.
D4	Named Entity Recognition identifies names of people, organizations, locations, dates, and other entities.
D5	TF-IDF represents text using term frequency and inverse document frequency weights.
D6	Cosine similarity measures similarity between numerical vectors.
D7	A question answering system retrieves an answer for a natural-language question.
D8	Text preprocessing may include lowercasing and punctuation handling.

System Architecture / Pipeline Design

Purpose: *Shows the end-to-end processing flow.*

`User question → Text corpus → Text normalization → TF-IDF vectors → Cosine similarity → Select best sentence → Display answer`

Pipeline modules:
- **Input:** Read a question from the user.
- **Corpus:** Store answer-bearing sentences.
- **Vectorization:** Convert the corpus and question into TF-IDF vectors.
- **Retrieval:** Compare vectors using cosine similarity.
- **Response:** Return the sentence with the highest score.

Backend Implementation

3.1 Approach / Algorithm

1. Store documents as strings.
2. Create a TfidfVectorizer.
3. Convert the documents into TF-IDF vectors.
4. Transform the user's question using the same vectorizer.
5. Calculate cosine similarity between the question and each document.
6. Return the sentence with the highest score.
7. If no useful terms match, display a no-answer message.

3.2 Python Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
  "NLP enables computers to understand human language.",
  "Tokenization splits text into smaller units called tokens.",
  "POS tagging assigns grammatical labels to words.",
  "Named Entity Recognition identifies people and locations.",
  "TF-IDF represents text using term frequency and inverse document frequency.",
  "Cosine similarity measures similarity between vectors.",
  "A question answering system retrieves an answer for a question."
]

vectorizer = TfidfVectorizer(stop_words="english")
doc_vectors = vectorizer.fit_transform(documents)

def answer_question(question):
```

```
    q_vector = vectorizer.transform([question])
    scores = cosine_similarity(q_vector, doc_vectors)[0]
    idx = scores.argmax()
    if scores[idx] == 0:
      return "No relevant answer found."
    return documents[idx]

question = input("Enter your question: ")
print("Answer:", answer_question(question))
```

User Interface / Execution

Run the code in Jupyter Notebook or Google Colab. Install the required library with `pip install scikit-learn`, execute the code, and enter a question such as “What does tokenization do?”

Sample Input & Output

Purpose: *Demonstrates expected system behaviour.*

Sample Question	Retrieved answer
What does tokenization do?	Tokenization splits text into smaller units called tokens.
What is NER used for?	Named Entity Recognition identifies people and locations.
Explain TF-IDF.	TF-IDF represents text using term frequency and inverse document frequency.
What is cosine similarity?	Cosine similarity measures similarity between vectors.

Results (Deliverable-Specific Outputs)

The program produces a TF-IDF representation of the corpus, calculates similarity scores for each sentence, and returns the top-ranked sentence. The actual score depends on the question and corpus wording.

Evaluation / Analysis

Purpose: *Checks retrieval quality using manually written questions.*

Sub-task	Observed / Expected Check	Remarks
Corpus preparation	7 educational sentences	Small corpus has limited topic coverage.
Vectorization	TF-IDF vectors created	Important words receive informative weights.
Ranking	Similarity score for each sentence	The highest score is selected.
Answer quality	Manually inspect returned sentence	Lexical overlap may not guarantee semantic correctness.

Interpretation

• TF-IDF and cosine similarity provide a simple, explainable retrieval baseline.
• It works best when the question and answer share important terms.
• Synonyms and paraphrases may reduce retrieval quality.
• A high score indicates textual similarity, not guaranteed correctness.

Challenges Faced & Solutions

Challenge 1: Different wording — Questions may use synonyms not found in the corpus. **Solution:** Add paraphrases or use semantic embeddings.
Challenge 2: Missing answer — The system may return a weak match. **Solution:** Use a minimum similarity threshold.
Challenge 3: Small corpus — Some topics are not covered. **Solution:** Add more reliable documents.

Conclusion

An information retrieval-based Question Answering system was developed using TF-IDF and cosine similarity. It compares a user's question with a small corpus and returns the most similar sentence. The approach is easy to understand, but its performance depends on the corpus coverage and shared vocabulary.

References

• scikit-learn Text Analytics: https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html
• TfidfVectorizer documentation:
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
• cosine_similarity documentation:
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

Project Output Screenshots

The following figures show output generated by the TF-IDF QA implementation.

```
In [1]: question = 'What does tokenization do?'

In [2]: answer, score = retrieve(question)


Out[2]:

Question: What does tokenization do?

Best match: Document D2

Answer: Tokenization splits text into smaller units called tokens.

Cosine similarity: 0.3867
```

Figure 1. Notebook-style display of actual question and retrieved answer.

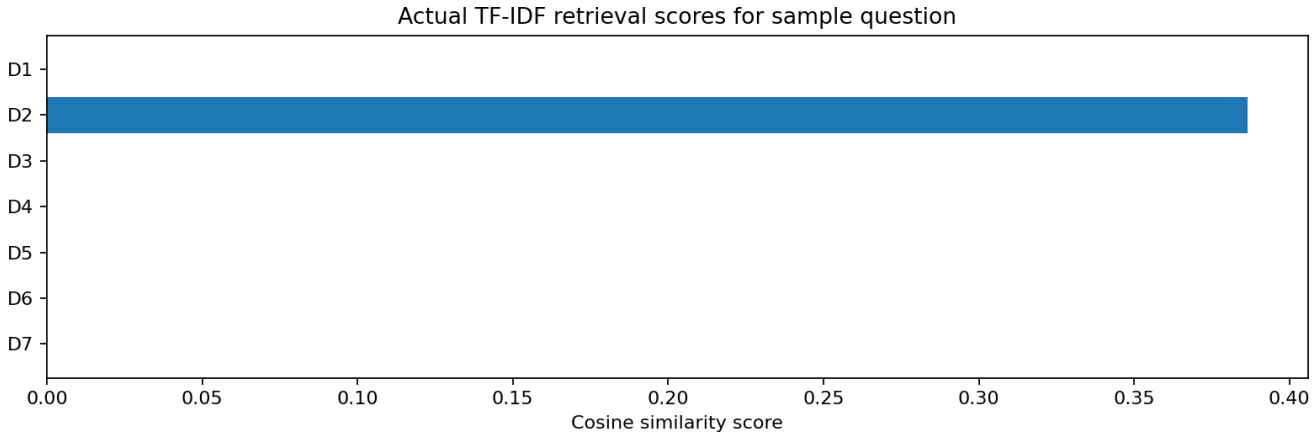


Figure 2. Actual cosine similarity scores across seven documents

Project Repository / Reproducibility

The experiment can be reproduced in a Python notebook by installing scikit-learn and Matplotlib, running the corpus preparation cell, fitting LDA, and then generating the topic-word output. No repository URL is claimed for this newly prepared report.

GitHub Repository URL	*https://github.com/vijwalsheshadri/Vijwal1GA23AI057-ANLP*
Repository Visibility	*Public*
Branch / Commit Used for Submission	*main*

System Pipeline Diagram

The architecture below illustrates the retrieval process used in the experiment.

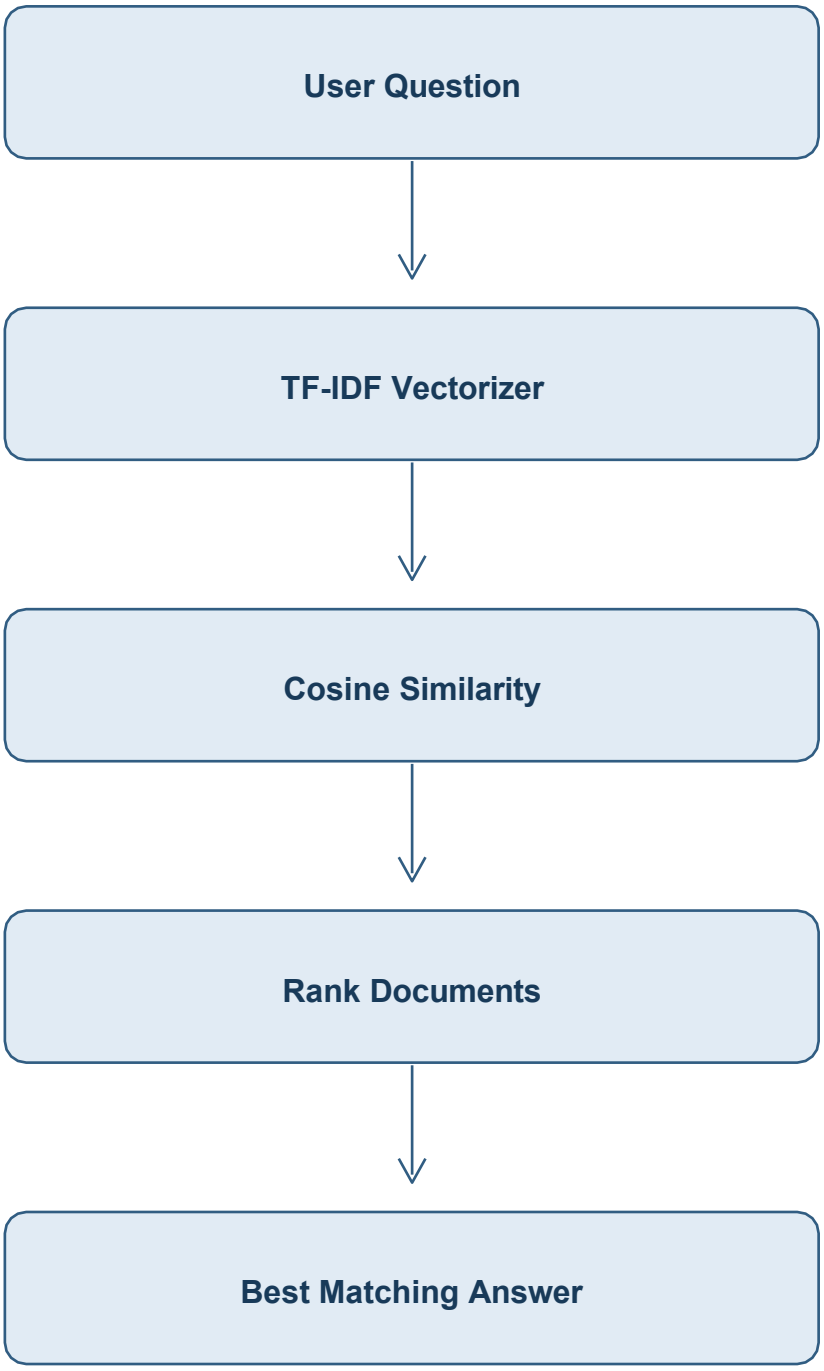


Figure 3. Information retrieval-based question answering pipeline.

ADVANCE NLP (AML23702)

LAB EXERCISE EVALUATION SHEET

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Lab Exercise No.	1
Experiment Title	Information Retrieval-Based Question Answering using TF-IDF and Cosine Similarity

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
	Total	50	

Overall Comments

Date of Submission	Signature of Faculty